Industr.IA

E-book

Operação, MLOps e ciclo de vida

Condução de Projetos | Unidade 5

Prof.: Raimundo Valter



SUMÁRIO

1. O ciclo de vida do modelo em produção 3
2. MLOps essencial 8
3. Débito técnico de ML 15
4. Conclusão 20

Unidade 5 | CDP-030

1. O ciclo de vida do modelo em produção

Este tópico abre o bloco mais denso do curso e tem por objetivo que você desenhe o ciclo de vida de um modelo em produção, com pontos de retreinamento, monitoramento de drift e critérios de pivô ou substituição explícitos (competência CP.B5.C1). O bloco retoma o caso corrente do curso, a manutenção preditiva dos compressores da planta química: o canvas de arquitetura do B3 fixou a solução, os dados e a inferência local na planta, e é sobre esse projeto já decidido que a pergunta deste bloco incide. A pergunta é direta e costuma pegar pesquisadores de surpresa: o que acontece com o modelo depois que ele funciona?

A premissa que organiza o bloco vem do contraste de posturas que o B1 estabeleceu. Um projeto de pesquisa e um projeto de PD&I industrial podem produzir modelos de qualidade comparável; a fase de modelagem não é o que os distingue. O que os distingue é o que acontece depois: a passagem do modelo treinado a um sistema operacional defensável, monitorado, versionado e mantido ao longo do tempo. Esse intervalo é justamente o que quem vem da pesquisa costuma desconhecer, por ausência de contato prévio com sistemas em operação. E é nele que projetos tecnicamente bons fracassam: não na métrica do teste, e sim no terceiro mês de operação, quando ninguém sabia que era preciso continuar olhando.

A primeira distinção operacional é entre entrega pontual e entrega continuada. A entrega pontual encerra o trabalho no momento em que o artefato é entregue e aceito: o relatório sai, o protótipo demonstra, a banca aprova. É o formato natural da pesquisa e de boa parte das consultorias. A entrega continuada assume que o artefato entregue é um sistema que seguirá operando sobre dados que mudam, e que a responsabilidade do time se estende pela operação: alguém monitora, alguém responde quando falha, alguém decide quando retreinar. A diferença não é retórica; ela muda o cronograma, o custo e a equipe. Um projeto de entrega continuada precisa orçar horas de operação recorrentes, definir quem as executa depois da residência e registrar isso no acordo com a empresa parceira desde o escopo.

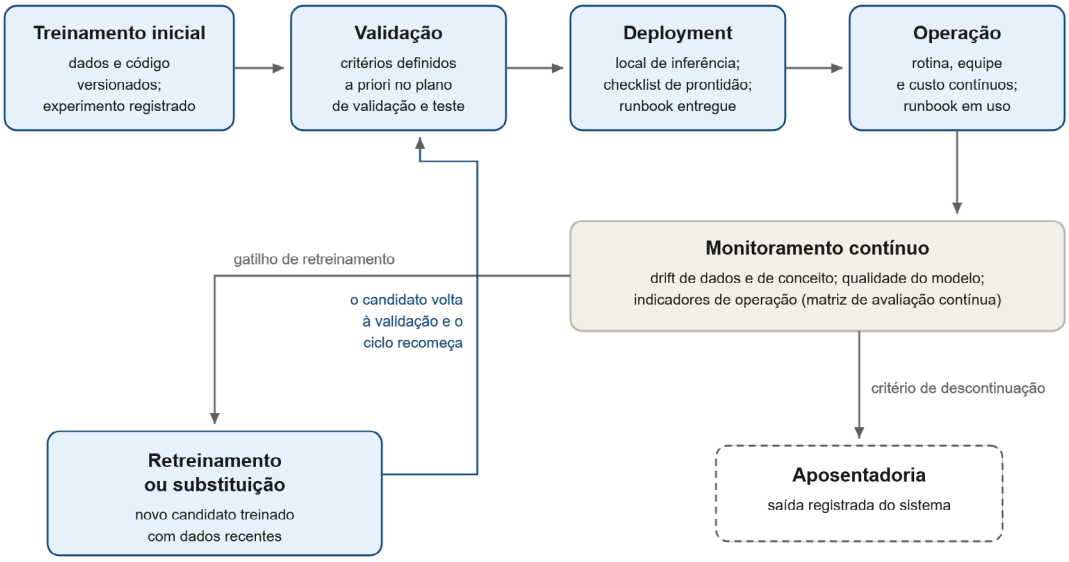
GLOSSÁRIO

Entrega continuada

Modalidade de entrega em que o resultado do projeto é um sistema que segue operando sobre dados que mudam, com responsabilidade declarada por monitoramento, resposta a falhas e atualização. Contrasta com a entrega pontual, que encerra o trabalho no aceite do artefato, e implica custo recorrente de equipe registrado desde o escopo.

Com a distinção posta, o ciclo de vida ganha forma. Este curso o organiza em seis fases: treinamento inicial, validação, deployment, operação, monitoramento e retreinamento ou substituição, com a aposentadoria do modelo como saída registrada do sistema. As fases não são um funil que se percorre uma vez; são um circuito. O monitoramento alimenta o retreinamento, o retreinamento produz um candidato que volta à validação, e o ciclo recomeça quantas vezes a vida útil do sistema exigir. A figura 1 apresenta o circuito completo, com os artefatos característicos de cada fase e as duas saídas do monitoramento.

Figura 1 - Ciclo de vida do modelo em produção, com as seis fases e as transições entre elas.



Fonte: autor. Arquivo: B5_fig-1_ciclo_de_vida_modelo_producao.svg

Para cego ver: Fluxograma intitulado "Figura 1 - Ciclo de vida do modelo em produção, com seis fases e as transições entre elas". O processo começa em Treinamento inicial, com dados e código versionados e experimento registrado; segue para Validação, com critérios definidos e plano de validação e teste; depois para Deployment, com local de inferência, checklist de prontidão e runbook entregue; e então para Operação, que envolve rotina, equipe, custos contínuos e runbook em uso. A partir da operação, o modelo passa por Monitoramento contínuo, que acompanha drift de dados e de conceito, qualidade do modelo e indicadores de operação. Se houver necessidade de atualização, um gatilho direciona o processo para Retreinamento ou substituição, com um novo candidato treinado usando dados recentes; esse candidato retorna à validação e o ciclo recomeça. Se forem atendidos critérios de descontinuação, o fluxo segue para Aposentadoria, com a saída do sistema devidamente registrada.

Leia a figura pelas transições, porque é nelas que mora a decisão. Do treinamento para a validação, a condição é reprodutibilidade: dados, código e configuração versionados, com o experimento registrado. Da validação para a implantação (deployment), a condição é o critério de corte definido antes: o candidato atinge, sobre dados que não viu, o que o plano de validação declarou como bom o suficiente. Do deployment para a operação, a condição é prontidão: checklist verificado, runbook entregue, responsáveis nomeados. E da operação em diante não há mais transição única: o monitoramento roda em paralelo e decide, pelos gatilhos declarados, se o ciclo volta ao retreinamento ou se o modelo caminha para a aposentadoria. Cada transição tem um artefato que a sustenta, e todos os quatro artefatos deste bloco aparecem na figura.

Entre a validação e a operação plena existe ainda um espaço intermediário que a prá-

tica industrial consolidou e que vale conhecer pelo nome. No shadow deployment, o candidato recebe o mesmo tráfego de inferência do modelo corrente e registra as suas predições de forma assíncrona, sem que ninguém as consuma: é validação com dado de produção e risco zero, ao custo de rodar dois modelos. No lançamento canário (canary deployment), o roteamento de tráfego entrega ao candidato uma fração pequena e reversível da carga real, e só a assume inteira depois de um período de observação sem incidente. Para o caso dos compressores, o squad registrou shadow deployment de quatro semanas antes de qualquer promoção: com falhas raras e caras, deixar o candidato provar-se em silêncio custa pouco e evita muito. O ponto de método é que a estratégia de exposição do candidato é uma decisão registrada no plano de validação e teste, e não um detalhe de implantação.

A validação merece uma pausa, porque é onde o arco iniciado no B3 se fecha pela primeira vez. A ficha de hipóteses e experimentos fixou, para cada hipótese, um critério de sucesso e um critério de descarte antes de qualquer resultado existir. O plano de validação e teste, artefato deste tópico, herda essa disciplina e a estende ao sistema: que conjuntos de dados servem de teste, que métricas correspondem aos critérios de sucesso técnico e de negócio da ficha de escopo, que resultado autoriza a promoção do candidato e que resultado obriga a voltar. Validar contra critério definido a priori é o que separa evidência de racionalização; validar contra o resultado que já se conhece é a forma mais educada de se enganar. No laboratório presencial, é essa mesma disciplina que o squad exercita no Encontro 4, ao preencher o Plano de Teste da Hipótese e consolidar a Matriz de Validação para o Desafio do seu eixo.

> **Exemplo aplicado**
>
> Uma empresa de bens de consumo contratou um modelo de previsão de demanda entregue como projeto pontual: relatório, código e uma apresentação final. Dezoito meses depois, o erro de previsão havia dobrado sem que ninguém percebesse, porque o mix de produtos e o canal de vendas mudaram e não havia monitoramento nem responsável. O prejuízo não veio de um modelo ruim, e sim de um modelo bom operando sem ciclo de vida: a empresa comprou uma entrega pontual quando o problema exigia uma entrega continuada.

A implicação de equipe merece ser dita sem eufemismo, porque é onde a entrega continuada cobra caro. Operar o sistema dos compressores exige, de forma recorrente: alguém que acompanhe os indicadores na cadência declarada, alguém que responda aos incidentes do runbook, alguém que conduza o retreinamento validado quando o gatilho dispara, e alguém do lado da planta que mantenha o servidor e o acesso aos dados. Durante a residência, esses papéis se distribuem no squad; depois dela, cada papel precisa de um destino declarado: transferência para a equipe da empresa parceira, contrato de suporte, ou descontinuação consciente. Projetos que não respondem a

essa pergunta no escopo respondem a ela no pior momento, quando o time se desfaz e o sistema continua rodando.

A fase de deployment traz de volta uma decisão que o projeto dos compressores já tomou: o local de inferência. O canvas do B3 registrou, pela lente Edge do curso base, que a inferência ocorre localmente, num servidor da própria planta: o dado de vibração não sai do ambiente da empresa e o alerta precisa caber na janela de reação da operação. Este bloco não reabre essa decisão; ele cobra o que ela custa em operação. Inferência local significa uma máquina que alguém da planta mantém, atualização de modelo que não é um clique numa console de nuvem e monitoramento que precisa alcançar um ambiente fora do alcance direto do time. O regime de repouso do dado, que motivou a decisão, é também o que define onde os logs ficam e quem pode lê-los.

Em outras palavras, cada ponto do continuum entre nuvem e borda compra um pacote de custos operacionais diferentes. Na nuvem, a atualização é simples e a escala é elástica, mas há custo recorrente por inferência, dependência de conectividade e dado saindo do ambiente da empresa. Na inferência local, em servidor da própria planta (on-premises), o dado repousa onde a regulação e o contrato pedem e a latência é curta e previsível, mas a equipe herda a manutenção da máquina, o procedimento de atualização e o alcance do monitoramento. Nenhum dos pacotes é errado; errado é escolher o local de inferência pela arquitetura e descobrir o custo operacional depois. No caso dos compressores, o custo da escolha local foi aceito e registrado: ele aparece no runbook, na matriz de avaliação contínua e no orçamento de operação.

Um desdobramento prático da inferência local merece registro próprio: o procedimento de atualização do modelo. Na nuvem, promover um candidato é uma operação de console; na planta, é um pequeno protocolo que precisa estar escrito antes da primeira atualização. O protocolo do caso registra quatro passos: o pacote do candidato chega com versão e soma de verificação; a instalação ocorre na janela combinada com a operação, nunca durante partida de equipamento; uma verificação pós-instalação confirma que o serviço responde e que as predições sobre um lote de referência batem com as da validação; e o pacote anterior permanece disponível para reversão imediata. É burocracia na medida certa: cinco linhas que transformam a atualização de um evento de risco em rotina, e que moram no runbook desde o primeiro deploy.

> **Armadilha comum**
> A armadilha de postura deste bloco é tratar a validação como linha de chegada: o modelo atingiu a métrica, o trabalho acabou, como um artigo aceito encerra uma pesquisa. Em PD&I industrial, a validação é o meio do caminho. A consequência de parar ali é um sistema órfão: sem monitoramento, sem responsável e sem critério de retreinamento, degradando em silêncio até falhar num momento caro. O engenheiro de PD&I planeja a operação no escopo, não depois do deploy.

A operação, quarta fase, é a mais longa e a menos glamourosa: é rotina. O sistema responde no prazo previsto, o pipeline de dados entrega no horário, os alertas chegam a quem opera os compressores, e alguém verifica tudo isso com cadência declarada. O instrumento da rotina é o runbook, que o tópico 2 detalha; o ponto deste tópico é de planejamento: rotina consome horas de gente com nome, e essas horas entram no custo do projeto. Um sistema cuja operação depende de boa vontade não tem operação; tem sorte, e sorte não passa em gate review.

O monitoramento, quinta fase, é o que transforma a avaliação pontual da pesquisa em avaliação contínua. A pergunta muda de o modelo é bom? para o modelo continua bom, aqui, neste trimestre, sobre estes dados? O instrumento que organiza essa pergunta é a matriz de avaliação contínua, artefato deste tópico: para cada indicador, ela registra o que monitorar, como medir, com que frequência, qual o gatilho de ação e quem é o destinatário da informação. Os indicadores se distribuem nas famílias que o tópico 2 detalha: drift, qualidade e operação. O desenho fino de como cada indicador chega a cada audiência, o que este curso chama de superfície do indicador, é assunto do bloco B6; aqui basta que cada linha da matriz tenha gatilho e destinatário declarados.

Uma linha preenchida torna a matriz concreta. Indicador: antecedência média do alerta em relação à falha confirmada. Como medir: comparação entre o horário do alerta e o registro de manutenção da falha correspondente. Frequência: apuração trimestral, porque falhas confirmadas são raras. Gatilho de ação: antecedência média abaixo da janela mínima que a operação declarou na ficha de escopo. Destinatário: o responsável técnico do sistema e o gestor de manutenção da planta. Repare no que a linha carrega: ela amarra um número de operação a um critério que veio do escopo e a duas pessoas com nome. Uma matriz cujas linhas não têm essa amarração é um painel bonito, não um contrato de monitoramento.

Vale explicitar a mudança de regime, porque ela é um dos objetivos declarados deste bloco: em produção, a avaliação contínua substitui a avaliação pontual; ela não a complementa. A avaliação pontual responde a uma pergunta datada, sobre um conjunto de teste congelado, e a resposta envelhece junto com os dados. Mantê-la como referência principal depois do deployment cria a ilusão confortável de um modelo aprovado de uma vez por todas. O que a operação exige é o contrário: a aprovação vence a cada janela de apuração, e o modelo permanece bom até que os indicadores digam o contrário. O conjunto de teste congelado ainda serve, mas para outra função: comparar candidatos entre si na validação, sob condições idênticas.

A última fase decide o destino do modelo, e a decisão precisa de critérios fixados antes, na mesma lógica dos critérios de pivô que a matriz de risco do B2 amarrou aos gates. Gatilho de retreinamento atingido, o time treina um candidato com dados recentes, e o

candidato volta à validação: retreinamento sem validação é troca de um modelo conhecido por um desconhecido. Quando o retreinamento deixa de recuperar o desempenho, ou quando o problema de negócio muda a ponto de invalidar a hipótese, a decisão sobe de nível: substituição por outra abordagem, que reabre o ciclo desde o treinamento, ou aposentadoria, com saída registrada, comunicação aos afetados e desligamento limpo das dependências. Aposentar um modelo sem registro é plantar um consumidor fantasma no sistema, um débito que o tópico 3 nomeia.

Fecham este tópico os seus dois artefatos. O plano de validação e teste registra conjuntos, métricas, critérios de promoção e de retorno, fazendo a ponte entre a fase de avaliação do método e a operação. A matriz de avaliação contínua registra os indicadores com frequência, gatilho e destinatário, e é o contrato do monitoramento. O caderno traz os dois preenchidos e comentados para o caso dos compressores, com a instrução de uso eixo a eixo. Resta saber com que ferramentas e com que práticas esse ciclo se sustenta no dia a dia: é o assunto do próximo tópico.

2. MLOps essencial

Este tópico trata dos fundamentos de MLOps essenciais para PD&I industrial e tem por objetivo que você aplique versionamento, pipelines, monitoramento, reprodutibilidade e runbook mínimo em escala compatível com o seu projeto (competência CP.B5.C2). A renúncia é declarada, como foi no B2 com a gestão de projeto: o curso não pretende formar engenheiro de MLOps especializado. O que um condutor de projeto precisa é do repertório mínimo para que o ciclo do tópico 1 funcione na prática, e para saber o que está delegando quando uma plataforma ou um especialista assume o resto.

MLOps é a aplicação, a sistemas de aprendizagem de máquina, da disciplina de operações que o software consolidou com DevOps: automatizar, versionar, monitorar e tornar reprodutível o caminho entre o código e o sistema em produção. A diferença é que um sistema de ML tem três fontes de mudança em vez de uma: o código, os dados e o modelo. Mudança em qualquer uma delas muda o comportamento do sistema, e é por isso que as práticas deste tópico tratam as três com o mesmo rigor. O princípio que as unifica é a reprodutibilidade, agora em chave operacional: não a reprodutibilidade de artigo, que permite a um par refazer o experimento, e sim a capacidade de reconstruir qualquer versão do sistema, com os dados e a configuração exatos daquela versão, no momento em que um gate, uma auditoria ou um incidente exigirem.

Um episódio recorrente mostra o que a reprodutibilidade operacional evita. Num gate, o time apresenta a métrica do candidato; um membro da banca pede a comparação com o resultado de seis semanas atrás; e a comparação não existe, porque o notebook mudou, o dado foi sobrescrito pela extração seguinte e ninguém registrou a semente

aleatória nem as versões do ambiente. O número antigo vira lenda: pode ter sido melhor, pode ter sido pior, ninguém pode afirmar. Nada disso exige ferramenta sofisticada para ser evitado; exige que cada resultado citável aponte para código, dados, configuração e ambiente recuperáveis. Em PD&I industrial, resultado que não se reconstrói não sustenta decisão, e um gate bem conduzido trata o número sem lastro como se não existisse.

A primeira prática é o versionamento, e a pergunta certa é o que versionar. A resposta operacional cobre cinco coisas, todas sob linhagem rastreável (data lineage): o código, os dados de treinamento, o modelo resultante, a configuração (hiperparâmetros, limiares e janelas) e o ambiente de execução. As cinco andam juntas: um modelo versionado sem a versão dos dados que o produziu é um número sem lastro, e um resultado sem a configuração que o gerou não se reproduz. O ganho é concreto e aparece em dois momentos: no gate review, quando a decisão de continuar ou pivotar precisa apontar para versões auditáveis, e no incidente, quando reverter para a última versão sã é a diferença entre uma noite ruim e uma semana ruim.

A segunda prática estende a primeira para onde os problemas de fato nascem: o pipeline de dados fica sob o mesmo regime de versionamento e monitoramento do pipeline de modelo. O B4 estabeleceu a governança da aquisição: acordo de dados com a empresa parceira, verificação de entrada a cada lote, dono e curador nomeados, rastreabilidade de transformações. Este tópico assume essa herança e a torna permanente: o pipeline que extrai, valida e transforma o dado dos sensores é parte do sistema em produção, versionado como o código e monitorado como o modelo. A assimetria de atenção é a armadilha: times monitoram o modelo com carinho e deixam o pipeline de dados solto, e então uma mudança silenciosa de esquema na fonte derruba a qualidade do modelo sem que nenhum indicador de modelo acuse a causa.

No caso dos compressores, o regime único tem forma concreta. A extração horária dos sensores é código versionado no mesmo repositório do modelo; cada lote submete-se a etapas de validação de qualidade dos dados (data quality checks), verificando restrições de esquema (schema validation), intervalos permitidos e taxa de preenchimento dos campos; o pipeline de engenharia de features é totalmente determinístico e rastreável do dado bruto ao vetor de características; e as métricas de saúde da ingestão (frequência de atualização, completude e volumetria) integram o painel de avaliação contínua que acompanha o modelo. Quando a planta trocar o firmware dos coletores, e ela vai trocar, a mudança será detectada na entrada do pipeline, e não na queda inexplicada da métrica três semanas depois.

A terceira prática é o pipeline de retreinamento, tratado aqui em alto nível e sem amarração a ferramenta. Dois desenhos dominam: o retreinamento agendado (schedule-based), por calendário, simples e previsível, e o retreinamento por evidência (event-driven),

disparado pelos gatilhos do monitoramento. O agendado paga treinamentos possivelmente desnecessários em troca de rotina; o por evidência paga a complexidade dos gatilhos em troca de reagir na hora certa. Em ambos, duas regras não se negociam: os dados de retreinamento passam pelas mesmas verificações de entrada do B4, e o candidato retreinado passa pela validação do tópico 1 antes de ser promovido. Retreinamento automático com promoção automática e sem validação é a receita documentada dos incidentes mais caros da área.

A quarta prática é o monitoramento em produção, e as famílias de indicadores merecem nomes precisos. Drift de dados é a mudança na distribuição das entradas em relação à distribuição do treinamento: os sensores recalibrados passam a reportar vibração em outra faixa, e o modelo, treinado na faixa antiga, opera fora do terreno que conhece. O drift de conceito é mais traiçoeiro, porque as entradas podem permanecer estáveis enquanto a relação entre entrada e saída muda. A terceira família é a qualidade do modelo propriamente dita, medida contra rótulos que em produção chegam com atraso: a falha que confirmaria o alerta de hoje só se registra semanas depois. A quarta é a operação: latência, disponibilidade, custo, saúde do pipeline.

GLOSSÁRIO

Drift de conceito

Mudança na relação entre as entradas e a saída que o modelo deveria prever, mesmo com a distribuição das entradas estável. Um novo regime de manutenção pode fazer com que os mesmos padrões de vibração deixem de anteceder falhas; o modelo continua vendo dados familiares e errando mais. Detecta-se pela queda de correspondência entre predição e desfecho.

A figura 2 organiza as quatro famílias na forma que a matriz de avaliação contínua adota: para cada família, o que ela observa, um exemplo concreto no caso dos compressores e o gatilho com a ação correspondente.

Figura 2 - Matriz de monitoramento e gatilhos de retreinamento, ilustrada com o caso dos compressores.

Família de indicador	O que observa	Exemplo no caso dos compressores	Gatilho e ação
Drift de dados	A distribuição das entradas mudou em relação à usada no treinamento	Deslocamento na distribuição de vibração após recalibração de um sensor	Desvio persistente na janela declarada: investigar a causa antes de retreinar
Drift de conceito	A relação entre entrada e saída mudou, mesmo com entradas estáveis	Alertas emitidos deixam de corresponder a falhas reais dos compressores	Queda de correspondência entre alerta e falha: rever a hipótese e retreinar com validação
Qualidade do modelo	A métrica da tarefa, calculada sobre rótulos que chegam com atraso	Precisão e antecedência dos alertas, apuradas por trimestre contra as falhas registradas	Métrica abaixo do critério do plano de validação: retreinar, ou levar ao gate a decisão de pivô
Operação	O sistema responde no prazo, no custo e na disponibilidade previstos	Latência da inferência local na planta; falhas do pipeline de dados dos sensores	Indicador fora da faixa: acionar o procedimento do runbook, com responsável nomeado

Fonte: autor. Arquivo: B5_fig-2_matriz_monitoramento_gatilhos.svg

Para cego ver: Tabela intitulada "Figura 2- Matriz de monitoramento e gatilhos de retreinamento, ilustrada com o caso dos compressores". A tabela possui quatro colunas: Família de indicador, O que observa, Exemplo no caso dos compressores e Gatilho e ação. Na linha Drift de dados, monitora-se se a distribuição das entradas mudou em relação aos dados de treinamento; o exemplo é o deslocamento na distribuição de vibração após a recalibração de um sensor, e um desvio persistente leva à investigação da causa antes do retreinamento. Em Drift de conceito, observa-se se a relação entre entrada e saída mudou; o exemplo é quando os alertas deixam de corresponder às falhas reais dos compressores, indicando necessidade de rever a hipótese e retreinar com validação. Em Qualidade do modelo, acompanha-se a métrica da tarefa, como precisão e antecedência dos alertas; resultados abaixo do critério definido podem acionar retreinamento ou decisão de pivô. Em Operação, verificam-se prazo, custo e disponibilidade do sistema; exemplos incluem latência da inferência e falhas no pipeline de sensores, que, quando fora da faixa esperada, acionam o procedimento definido no runbook.

Dois cuidados de leitura evitam os erros mais comuns com essa matriz. O primeiro: gatilho sem janela declarada não é gatilho, é opinião; desvio persistente por quantas horas, queda de quanto, medida sobre qual período, tudo isso se escreve antes. O segundo: drift detectado não implica retreinamento imediato. O primeiro passo é investigar a causa, porque o desvio pode ser um defeito de sensor, uma mudança de esquema no pipeline ou um evento transitório da planta; retreinar sobre dado defeituoso ensina o defeito ao modelo. O retreinamento responde a drift real e persistente, e o incidente de dado responde ao runbook.

Há ainda o problema prático que a terceira família carrega: em produção, o rótulo verdadeiro chega tarde ou não chega. A falha que confirmaria o alerta pode levar semanas para ocorrer, e cada intervenção bem-sucedida a impede de ocorrer. Enquanto o rótulo não chega, o monitoramento se apoia em indicadores substitutos: a distribuição das predições (um modelo que passa a alertar o dobro mudou, ainda que não se saiba se para melhor), a taxa de alertas confirmados pela inspeção de campo, o retorno estruturado da equipe de manutenção sobre cada alerta atendido. Indicador substituto não substitui a métrica da tarefa; ele compra tempo de reação enquanto ela não pode ser calculada, e a matriz de avaliação contínua registra os dois com papéis distintos.

A quinta prática fecha o arco aberto no B3: o registro de experimentos. A ficha de hipóteses e experimentos fixou critérios de sucesso e de descarte com prazo fechado; o registro de experimentos é onde cada execução deposita seus parâmetros, seus dados, seu código e suas métricas, e portanto é onde cada hipótese recebe veredito auditável. Sem registro, o veredito vira memória de reunião: alguém lembra que a alternativa B foi testada e não deu certo, ninguém sabe com que dados nem contra que critério. Com registro, a cadeia hipótese, experimento, resultado e decisão se sustenta em evidência que um gate, um mentor ou um auditor podem percorrer. É a diferença, mais uma vez, entre conduzir e torcer.

O conteúdo mínimo de cada registro cabe numa linha de disciplina: identificador do experimento, hipótese da ficha a que responde, versão dos dados, versão do código, configuração completa, métricas contra os critérios declarados, e a decisão tomada com uma frase de justificativa. O custo de registrar é de minutos; o custo de não registrar aparece no gate, quando a cadeia hipótese, experimento, alternativa e decisão que o B3 montou precisa ser percorrida de trás para a frente. E há um ganho menos óbvio: o registro protege o time de si mesmo, porque impede que a alternativa descartada com critério volte à mesa três semanas depois por insistência de memória seletiva. Descarte registrado é descarte que fica descartado, salvo evidência nova.

Armadilha comum

Tratar AutoML como substituto dos critérios do projeto. A busca automatizada de modelos e hiperparâmetros é útil para explorar alternativas com prazo fechado, mas ela otimiza a métrica que recebe, não o critério que importa. No caso dos compressores, um AutoML maximizando acurácia entregaria um modelo cego ao custo assimétrico de perder uma falha real. Os critérios da ficha de hipóteses continuam mandando, e cada execução do AutoML entra no registro de experimentos como qualquer outra.

A sexta prática é a orquestração dos pipelines, e este curso a trata como decisão de responsabilidade, não como escolha de ferramenta. Três perguntas a resolvem. Que pipeline precisa de orquestração? O que roda recorrentemente, com dependências e sem uma pessoa olhando: a extração diária dos sensores, o retreinamento mensal, o cálculo dos indicadores. De quem é a responsabilidade operacional quando o pipeline falha de madrugada? Um nome, não um time abstrato; e o nome muda quando a residência termina. E o que disso entra nos artefatos? O procedimento de resposta entra no runbook; a existência de alerta de falha e de responsável nomeado entra no checklist de prontidão. Ferramenta que orquestra sem responsável declarado só transforma falha silenciosa em falha barulhenta e igualmente órfã.

A decisão simétrica também é legítima e igualmente registrável: não orquestrar. Num projeto pequeno, com dois pipelines e um time presente, um agendador simples do sistema operacional, ou mesmo a execução manual documentada com checklist e cadência, pode ser a escolha certa; a orquestração dedicada compra robustez ao preço de mais uma peça de infraestrutura que alguém precisa manter. O erro não está em nenhum dos dois lados; está em não decidir. Pipeline recorrente sem decisão registrada sobre quem o executa e como se sabe que ele falhou é o estado que este bloco não admite, com ou sem ferramenta.

Exemplo aplicado

No caso dos compressores, o squad mapeou três pipelines recorrentes: a ingestão horária dos sensores, o cálculo diário dos indicadores de drift e o retreinamento mensal. A ingestão foi orquestrada com alerta imediato, porque uma hora de dado perdido compromete o alerta de falha; o responsável na planta ficou nomeado no runbook, com o engenheiro de dados do squad como escalada. O retreinamento ficou agendado, mas com promoção manual: o candidato só substitui o modelo após a validação, decidida por uma pessoa com nome.

Resta a pergunta inevitável: com que ferramentas? A resposta deste curso tem duas camadas. A primeira é de fronteira: o curso de Programação para IA mostra o ambiente, a manipulação de dados e a chamada a APIs de modelo, mas não cobre MLflow, orquestradores nem ferramentas de monitoramento; a curadoria, portanto, fica aqui, e fica em alto nível. A segunda é de postura: a escolha de ferramenta é condicionada pelo stack da empresa parceira, pelo custo e pela responsabilidade operacional, e não pela moda. O quadro 1 organiza as categorias com exemplos representativos; nomes citados são referência de categoria, não prescrição.

Quadro 1 - Curadoria de ferramentas de MLOps essencial, em alto nível.

Categoria	Ferramentas representativas	O que resolve	Quando entra
Versionamento de dados e modelo	DVC, lakeFS	Amarra cada versão do modelo à versão exata dos dados e do código que a produziu	Do primeiro experimento em diante
Registro de experimentos	MLflow, Weights and Biases	Registra parâmetros, métricas e artefatos de cada execução, dando veredito auditável às hipóteses	Do primeiro experimento em diante
Orquestração de pipelines	Airflow, Prefect	Executa pipelines recorrentes com dependências, retentativas e alerta de falha	Quando um pipeline passa a rodar sem pessoa olhando
Monitoramento em produção	Evidently, Grafana	Acompanha drift, qualidade e operação e dispara os gatilhos da matriz de avaliação contínua	Do deployment em diante

Para cego ver: Quadro 1 intitulado "Curadoria de ferramentas de MLOps essencial, em alto nível". A tabela possui quatro colunas: Categoria, Ferramentas representativas, O que resolve e Quando entra. Em Versionamento de dados e modelo, aparecem DVC e lakeFS, usados para vincular cada versão do modelo à versão exata dos dados e do código que a produziu, desde o primeiro experimento. Em Registro de experimentos, aparecem MLflow e Weights and Biases, que registram parâmetros, métricas e artefatos de cada execução, permitindo auditoria das hipóteses, também desde o primeiro experimento. Em Orquestração de pipelines, aparecem Airflow e Prefect, responsáveis por executar pipelines recorrentes com dependências, novas tentativas e alertas de falha, sendo úteis quando o pipeline passa a rodar sem acompanhamento constante. Em Monitoramento em produção, aparecem Evidently e Grafana, que acompanham drift, qualidade e operação do sistema, acionando gatilhos da avaliação contínua a partir do deployment.

As duas últimas práticas produzem os dois artefatos restantes do bloco. O runbook mínimo é o documento que permite operar o sistema sem depender da memória de quem o construiu. A estrutura deste curso pede quatro seções: a identificação do sistema, com versões correntes e dependências; os procedimentos operacionais, cada um no formato quando acontece X, faça Y, com ao menos três documentados (falha do pipeline de ingestão, drift persistente detectado, modelo indisponível no servidor da planta); os responsáveis com contato e cadeia de escalada; e os gatilhos que disparam cada procedimento, coerentes com a matriz de avaliação contínua. O teste de qualidade de um runbook é impessoal: uma pessoa competente que nunca viu o projeto consegue responder ao incidente seguindo o que está escrito.

Um procedimento escrito mostra o padrão esperado. Quando o painel indicar ausência de dados de um compressor por mais de duas horas: primeiro, verificar no log de ingestão se a falha é do coletor local ou da transmissão; segundo, acionar o contato de automação da planta pelo canal registrado, com o identificador do compressor; terceiro, registrar o incidente com horário de início; quarto, se a ausência passar de oito horas, suspender os alertas daquele compressor e comunicar o gestor de manutenção, porque alerta calculado sobre dado ausente é pior que ausência de alerta. Repare nos traços do padrão: gatilho objetivo, passos ordenados, contatos apontados, e uma decisão de segurança tomada de antemão, para não ser improvisada às três da manhã.

O checklist de prontidão para produção é o gate material entre o deployment e a operação: uma lista de verificações objetivas que precisam estar demonstráveis antes de o sistema assumir responsabilidade real. Os itens cobrem o que este bloco construiu: versões amarradas e reproduzíveis, validação aprovada contra critério a priori, monitoramento ativo com gatilhos e destinatários declarados, runbook entregue com responsáveis nomeados, pipeline de dados sob verificação de entrada, acordo de dados vigente. Cada item é verificável por evidência, no mesmo espírito do critério de pronto do B2: prontidão declarada sem demonstração é a versão operacional da promessa que o B4 ensinou a desconfiar.

Falta dizer quem verifica. O checklist de prontidão é mais eficaz quando a verificação não é feita por quem construiu o item verificado: no squad, isso significa trocar os pares, com quem cuidou do modelo verificando o pipeline e vice-versa; na relação com a empresa parceira, significa percorrer os itens que a tocam, como o acordo de dados e o acesso ao servidor, junto com o responsável do lado dela. A verificação cruzada custa uma reunião e paga em duas moedas: encontra o item que o autor não vê mais por familiaridade, e transforma a prontidão numa afirmação coletiva, que ninguém precisa sustentar sozinho quando o sistema assumir carga real.

Para aprofundar

HUYEN, C. Designing Machine Learning Systems. O'Reilly, 2022. Os capítulos 8 a 10 cobrem monitoramento de shifts de distribuição, aprendizado contínuo e teste em produção, e infraestrutura de MLOps, na profundidade que este bloco defere. A lista curada da mesma autora, em huyenchip.com/mlops, organiza o caminho de estudo por nível.

O caderno traz o checklist de prontidão e o runbook mínimo preenchidos e comentados para o caso dos compressores, com a instrução de uso item a item. Com o ciclo desenhado e as práticas que o sustentam no lugar, falta encarar uma verdade incômoda: mesmo um sistema bem operado acumula, pelo próprio uso de aprendizagem de máquina, um custo escondido que cresce em silêncio. Esse custo tem nome, tipologia e literatura clássica, e é o assunto do terceiro tópico.

3. Débito técnico de ML

Este tópico fecha o bloco e tem por objetivo que você identifique as cinco formas de débito técnico de ML em um cenário apresentado e proponha mitigações coerentes (competência CP.B5.C3). A referência é o paper que fundou essa conversa: Hidden Technical Debt in Machine Learning Systems, de Sculley e colegas, publicado no NeurIPS em 2015 a partir da experiência de sistemas em produção no Google. O ambiente tecnológico mudou desde então, e por isso este bloco preserva a tipologia e atualiza os exemplos; a tese central, porém, envelheceu bem: os ganhos rápidos de ML não vêm de graça, e o custo escondido se paga em manutenção.

A metáfora do débito vem da engenharia de software: uma solução expediente hoje cobra juros em manutenção amanhã, e a decisão de contraí-la pode até ser correta, desde que consciente. O que Sculley e colegas mostraram é que sistemas de ML contraem formas de débito que o software tradicional não conhece, porque o comportamento do sistema passa a depender dos dados, e dados não têm as fronteiras limpas do código. Este curso organiza essas formas em cinco categorias: erosão de fronteiras, emaranhamento, inferno de dependências, feedback loops e débito de configuração.

GLOSSÁRIO

Débito técnico de ML

Custo futuro de manutenção contraído pelas escolhas de construção de um sistema de aprendizagem de máquina, somando ao débito clássico de software formas específicas de ML: erosão de fronteiras, emaranhamento, dependências de dados, feedback loops e configuração espalhada. Não se elimina; gerencia-se conscientemente ao longo do ciclo, com momentos declarados de pagamento.

A erosão de fronteiras é a forma mais estrutural. A engenharia de software vive de fronteiras de abstração: cada módulo tem um contrato, e mudar um não deveria quebrar os outros. Um modelo de ML dissolve essas fronteiras, porque o seu comportamento não é declarado num contrato, é aprendido dos dados. Dois sintomas caracterizam a erosão de fronteiras. A correção em cascata (cascade correction): quando, para mitigar erros do modelo A, acopla-se um modelo B que consome o output de A como feature, e posteriormente um modelo C sobre B, criando um grafo de dependências rígido e suscetível à propagação de erros. O outro sintoma é o consumidor não declarado (undeclared consumer): sistemas a jusante que consomem a predição do modelo via logs ou tabelas sem um contrato de API ou esquema formal; quando o modelo muda, um processo que ninguém mapeou quebra junto.

O emaranhamento é a erosão vista de dentro do modelo, e Sculley e colegas lhe deram uma sigla que vale decorar: CACE, changing anything changes everything. Em modelos não lineares e multivariados, as variáveis de entrada não apresentam independência condicional; o algoritmo aprende distribuições de probabilidade conjuntas sobre todo o espaço de features. Alterar a distribuição de uma única feature, remover um atributo ou recalibrar um sensor altera a importância relativa das demais variáveis e redistribui os pesos do modelo, com efeito longe do ponto da mudança. A consequência prática é que não existe mudança pequena nas entradas de um modelo em produção, e é por isso que a verificação de entrada do B4 e o monitoramento por feature deste tópico não são luxo, são o preço do emaranhamento.

O inferno de dependências tem duas metades, e a mais cara é a menos visível. A metade de código é conhecida: bibliotecas, versões, ambientes. A metade de dados é a novidade do ML: o sistema depende de sinais que outros produzem, e a observação central do paper é que dependências de dados custam mais que dependências de código, porque não há compilador que as verifique. Uma dependência de dados instável muda de comportamento sem avisar; uma subutilizada agrega risco sem agregar valor. A versão contemporânea dessa forma é a dependência de um modelo de terceiro atrás de uma API: o provedor atualiza a versão, o comportamento muda, e o sistema que consumia a saída herda a mudança sem ter mudado uma linha. Quem cursou o B3 reconhece aqui o mapa de fornecedores e PI operando como instrumento de prevenção.

Os feedback Loops aparecem quando as predições do sistema alteram os dados que o sistema verá no futuro. O loop direto é visível: um recomendador que só mostra o que prevê que agrada coleta cliques apenas sobre o que mostrou, e confirma a si mesmo. O loop oculto passa por fora do sistema: dois modelos que não se conhecem interagem pelo mundo, um alterando o ambiente que treina o outro. A manutenção preditiva é vulnerável ao loop direto, que gera viés de seleção (selection bias) nos dados de treino. Quando as intervenções guiadas pelo modelo evitam as falhas, os eventos de falha ra-

reiam na distribuição de classes do conjunto futuro, e o retreinamento perde exatamente os exemplos de que precisa para manter a capacidade discriminativa do modelo. O sucesso do sistema empobrece o seu próprio alimento, e quem olha só a taxa de falhas pode confundir degradação com vitória.

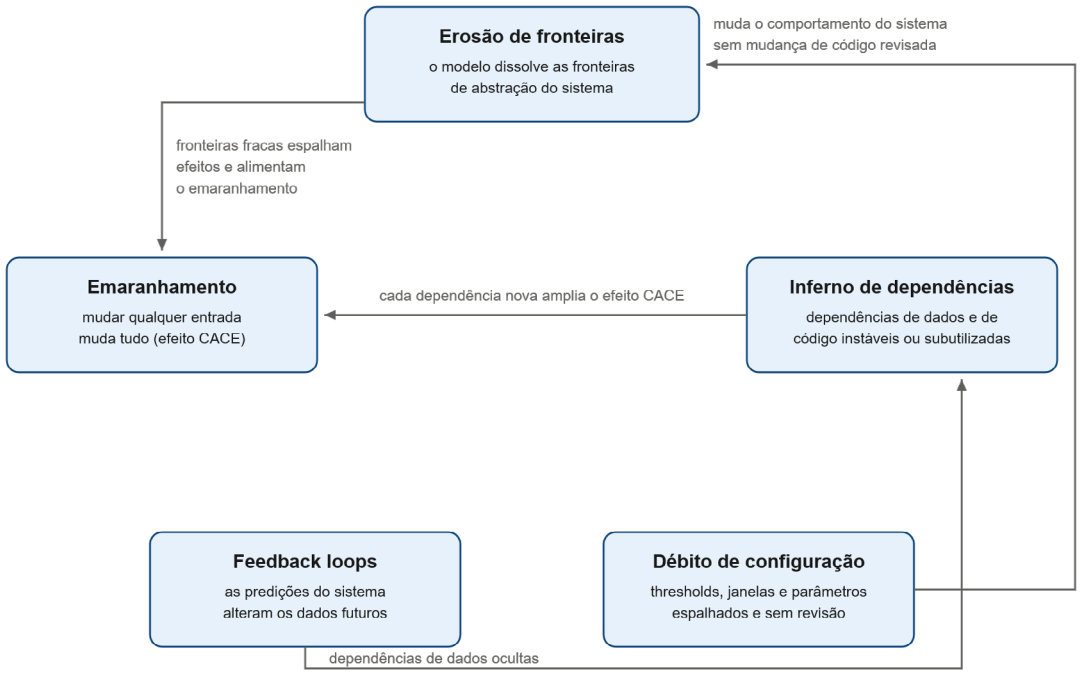
O loop oculto tem versão industrial igualmente plausível. Suponha que o agendamento de produção da planta, um sistema que o squad não conhece, passe a usar os alertas de manutenção para redistribuir carga entre compressores. A redistribuição muda o perfil de operação dos equipamentos, que muda os sinais de vibração, que muda o comportamento do modelo, que muda os alertas que o agendamento consome. Dois sistemas que não se conhecem, acoplados pelo mundo físico, cada um treinando o ambiente do outro. Não há como proibir esse acoplamento; há como torná-lo visível, e é para isso que o mapa de consumidores das predições existe.

O débito de configuração é o mais prosaico e, por isso, o mais subestimado. Um sistema de ML maduro carrega mais linhas de configuração do que de código: thresholds de alerta, janelas de agregação, listas de features, versões apontadas, flags de experimento. Quando essa configuração vive espalhada, em planilhas, variáveis de ambiente e valores fixados no código, ela muda o comportamento do sistema sem passar por revisão nenhuma, e ninguém sabe dizer qual configuração produziu qual resultado. A versão contemporânea inclui prompts e limiares de decisão de modelos generativos: texto que é, na prática, configuração de produção, editado com a informalidade de um rascunho. O tratamento é conhecido: configuração versionada como código, revisada como código e amarrada ao registro de experimentos.

Ao lado das cinco formas, o paper nomeia anti-padrões de sistema que funcionam como sinais visíveis do débito invisível. O código de integração (glue code): a maior parte da base de código não consiste nos algoritmos de ML, mas em código utilitário de suporte para ingestão, ETL, formatação de payloads e chamadas de API, frequentemente negligenciado em testes unitários e de integração. A selva de pipelines (pipeline jungle): orquestrações de dados que se acumulam organicamente por acréscimo de etapas de pré-processamento sem refatoração porque não se sabe o que consome cada saída. E os caminhos experimentais mortos: flags e desvios de experimentos antigos que permanecem no código como armadilhas armadas. Nenhum desses padrões derruba o sistema hoje; todos encarecem cada mudança futura, e a limpeza deles é exatamente o tipo de pagamento de débito que a revisão de processo dos gates agenda.

As cinco formas raramente aparecem isoladas, e a figura 3 mapeia as interações típicas: fronteiras fracas alimentam o emaranhamento, cada dependência nova amplia o efeito CACE, feedback loops criam dependências de dados ocultas, e a configuração espalhada muda o sistema por baixo de todas as outras.

Figura 3 - Mapa das cinco formas de débito técnico de ML e suas interações típicas.



Fonte: autor. Arquivo: B5_fig-3_mapa_debito_tecnico_ml.svg

Para cego ver: Diagrama intitulado "Figura 3- Mapa das cinco formas de débito técnico de ML e suas interações típicas". O esquema apresenta cinco blocos conectados por setas. No topo está Erosão de fronteiras, quando o modelo dissolve as fronteiras de abstração do sistema. À esquerda aparece Emaranhamento, indicando que mudar uma entrada pode afetar todo o sistema, conhecido como efeito CACE. À direita está Inferno de dependências, relacionado a dependências de dados e de código instáveis ou pouco utilizadas. Na parte inferior esquerda aparece Feedback loops, em que as previsões do sistema alteram os dados futuros. Na parte inferior direita está Débito de configuração, associado a thresholds, janelas e parâmetros espalhados e sem revisão. As setas mostram como esses problemas podem se reforçar: fronteiras frágeis ampliam o emaranhamento; novas dependências aumentam o efeito CACE; configurações podem contribuir para mudanças no comportamento do sistema e para dependências ocultas; e essas interações tornam o débito técnico de sistemas de machine learning mais complexo e interdependente.

Exemplo aplicado

Num varejista online, o modelo de recomendação da vitrine passou a alimentar, sem que o time soubesse, o modelo de compras que decide reposição de estoque: um consumidor não declarado. Quando a vitrine mudou de estratégia, a demanda prevista deslocou, o estoque encalhou, e a investigação levou semanas porque o vínculo entre os dois sistemas não constava de mapa nenhum. Um feedback loop oculto somado a uma dependência de dados não declarada: duas formas de débito interagindo, nenhuma visível no código.

A mitigação se organiza por categoria, e quase toda ela já foi construída ao longo do curso; o que este tópico faz é nomear a que débito cada prática responde. Contra a erosão e o consumidor não declarado: contratos de dados explícitos e o mapa de quem consome cada saída, na linha do catálogo do B4. Contra o emaranhamento: verificação de entrada, monitoramento por feature e retreinamento sempre validado. Contra as dependências: o inventário de fornecedores e PI do B3 estendido aos sinais de dados, com versão fixada e revisão periódica do que é subutilizado. Para mitigar os feedback loops: registrar estruturadamente as intervenções operacionais como metadados do pipeline de dados e manter um grupo de controle não exposto às predições do modelo (holdout group ou amostragem aleatória não intervencionista) para obter a taxa real não enviesada de eventos de falha. Contra o débito de configuração: configuração como código, versionada e revisada. Nada disso zera o débito; tudo isso o torna visível e cobrável.

O caso comentado do bloco aplica a tipologia ao projeto dos compressores, e três formas aparecem com nitidez. A primeira é o feedback loop direto: o sistema existe para provocar intervenções antes da falha, logo cada acerto remove do futuro um exemplo de falha; a mitigação registrada foi tratar toda intervenção como evento de dados, com causa e achado de campo anotados pela manutenção, e acompanhar a taxa base de falhas por trimestre para distinguir melhoria real de cegueira progressiva. A segunda é o emaranhamento: a recalibração de um sensor de vibração, mudança local e legítima da planta, desloca uma feature e redistribui o comportamento do modelo inteiro; a mitigação foi amarrar o calendário de calibração da planta à verificação de entrada e ao indicador de drift de dados, com retreinamento validado quando o desvio persiste.

A terceira forma é o débito de configuração: os thresholds de alerta por compressor, as janelas de agregação dos sinais e a lista de features viviam em três lugares diferentes ao final da prototipagem; a mitigação foi consolidar tudo em configuração versionada junto do modelo, com cada mudança registrada e apontada no runbook. Vale notar o que o caso não mostra: nenhuma das três mitigações exigiu ferramenta nova ou reescrita; exigiram nomear o débito, decidir conscientemente o que pagar agora e o que carregar, e registrar a decisão onde o gate a encontra. As duas formas restantes seguem no radar do caso, com sinais de alerta declarados. Para o inferno de dependências, o sinal é a biblioteca de extração de sinais fornecida pelo fabricante dos sensores: versão fixada, changelog acompanhado e teste de regressão dos indicadores a cada atualização. Para a erosão de fronteiras, o sinal é o interesse crescente do planejamento de produção nos alertas do sistema: antes que a predição apareça numa planilha de terceiros, o consumo vira contrato declarado, com aviso de mudança. O caso comentado do caderno percorre as cinco formas com esse mesmo padrão: sinal de alerta, evidência e mitigação com dono.

Para aprofundar

SCULLEY, D. et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS, 2015. O paper original, curto e legível, com as formas de débito e os anti-padrões de sistema; a leitura direta compensa. Disponível em papers.nips.cc/paper/5656.

Fica, por fim, a postura, que é o que este bloco quer formar. Débito técnico de ML não é falha moral nem acidente evitável; é propriedade estrutural de sistemas cujo comportamento depende de dados, e a pergunta profissional não é como zerá-lo, e sim quanto dele estamos carregando, onde, e quando pagamos. A revisão de processo dos gates, que o B2 instituiu, é o momento natural dessa pergunta: a cada gate, o time nomeia o débito acumulado no ciclo e decide o que paga antes do próximo. Gerenciar débito conscientemente é a última fronteira entre operar um sistema e ser operado por ele.

Um último encaixe fecha a tópico com o vocabulário do curso: débito técnico identificado é um risco operacional como os que a matriz do B2 organiza, e recebe o mesmo tratamento. O feedback loop do caso, por exemplo, entra na matriz de risco com probabilidade, severidade, mitigação com responsável e gatilho de reavaliação a cada gate; se a taxa base de falhas cair abaixo do limiar que impede retreinamento confiável, o risco vira pauta de decisão, não surpresa. A tipologia de Sculley e colegas não pede um instrumento novo; pede que o instrumento que o curso já deu enxergue uma família de riscos que quem vem da pesquisa não aprendeu a ver.

4. Conclusão

Os três tópicos deste ebook cobriram o intervalo que mais distingue o engenheiro de PD&I industrial do pesquisador: o que acontece depois que o modelo funciona. O tópico 1 desenhou o ciclo de vida em produção: a distinção entre entrega pontual e entrega continuada, as seis fases com seus critérios de transição, o custo operacional do local de inferência que o canvas do B3 fixou, e a avaliação contínua no lugar da avaliação pontual. O tópico 2 entregou o MLOps essencial: versionamento das três fontes de mudança, o pipeline de dados sob o mesmo regime do pipeline de modelo, monitoramento com gatilhos e janelas declarados, registro de experimentos como veredito auditável das hipóteses do B3, orquestração como decisão de responsabilidade, runbook e checklist de prontidão. O tópico 3 nomeou o custo escondido: as cinco formas de débito técnico de Sculley e colegas, suas interações, e a postura de gerenciá-lo conscientemente em vez de fingir que não existe.

O ganho do bloco é o fechamento de um arco que começou dois blocos atrás. As hipóteses que o B3 formulou com critérios a priori recebem veredito auditável no registro de experimentos; a arquitetura e o local de inferência que o canvas fixou cobram agora o seu custo operacional declarado; a validação herda os critérios e os estende ao sistema; e a governança de dados do B4 vira regime permanente de operação. Nenhum artefato deste bloco nasce do zero: cada um lê os anteriores, e é essa cadeia de coerência que uma empresa parceira, um mentor ou uma banca percorrem quando perguntam se o sistema é operável. Na residência, o runbook iniciado aqui é o ponto de partida do runbook efetivo do projeto, insumo direto para a empresa parceira depois dos seis meses.

Antes de avançar, percorra no caderno os quatro artefatos deste bloco (plano de validação e teste, matriz de avaliação contínua, checklist de prontidão para produção e runbook mínimo), que trazem o preenchimento comentado para o cenário de exemplo e a instrução de uso eixo a eixo. Feche o bloco com o quiz formativo. O bloco B6 começa onde este termina: com o sistema em operação e o ciclo registrado, entra a comunicação, e você aprenderá a relatar progresso e incerteza para audiências distintas, a defender decisões de arquitetura e operação diante de audiência crítica, e a produzir o model card e o datasheet que refletem o que o ciclo registrou.

REFERÊNCIAS

Bibliografia mínima

HUYEN, C. Designing Machine Learning Systems. O'Reilly, 2022. Referência central do bloco. Capítulos 8 (Data Distribution Shifts and Monitoring), 9 (Continual Learning and Test in Production), 10 (Infrastructure and Tooling for MLOps) e 11 (The Human Side of Machine Learning).

SCULLEY, D. et al. Hidden Technical Debt in Machine Learning Systems. NeurIPS, 2015. Referência incontornável sobre débito técnico de ML, tratada centralmente na tópico 3. Disponível em papers.nips.cc/paper/5656.

HUYEN, C. MLOps guide. Lista curada de materiais sobre MLOps, de fundamentos a estudos de caso, organizada por nível. Disponível em huyenchip.com/mlops.

Bibliografia complementar

RENGGLI, C. et al. Continuous Integration of Machine Learning Models with ease.ml/ci: Towards a Rigorous Yet Practical Treatment. SysML (MLSys), 2019. Preprint em arXiv.org/abs/1903.00278.

SAMBASIVAN, N. et al. Everyone Wants to Do the Model Work, Not the Data Work: Data Cascades in High-Stakes AI. CHI, 2021. Referência transversal sobre o subinvestimento em trabalho de dados e operação, mencionada também no bloco B4.

ZINKEVICH, M. Rules of Machine Learning: Best Practices for ML Engineering. Google. Disponível em developers.google.com/machine-learning/guides/rules-of-ml.

GAMA, J. et al. A Survey on Concept Drift Adaptation. ACM Computing Surveys, v. 46, n. 4, 2014. DOI 10.1145/2523813. Tratamento sistemático de drift de dados e de conceito.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.